

Motion-Window Conditioning for Embodiment-Aware Multi-Humanoid Motion Tracking

Kevin Händler¹, Ivan Smirnov¹, Maryana Smirnova¹, Nico Bohlinger^{1,2}, and Michael Drolet^{1,2}

Abstract—Tracking diverse whole-body motions across different humanoids requires a controller to account for both motion and embodiment. We extend a shared, descriptor-conditioned per-joint policy to track retargeted motion on five simulated humanoid models, with body randomization during training. We compare current joint targets with richer motion windows, supplied directly or through finite scalar quantization (FSQ). Under the tested training curriculum, window-conditioned policies avoid an observed upper-body tracking instability. Raw and quantized windows achieve comparable accuracy over matched budgets, showing that this improvement does not require quantization. The FSQ representation instead provides a compact command interface that supports control without a direct joint-reference input and remains usable at reduced command-update rates. Evaluation includes held-out motion-capture sequences and exploratory cross-body tests. Lower-body tracking, heading accuracy and general transfer to unseen bodies remain limitations.

Supplementary videos: <https://humanoid-tracking-videos.pages.dev/>

I. INTRODUCTION

Humanoid control must scale along two axes: motion and embodiment. A larger motion set requires one policy to represent different poses, timings and transitions. A larger embodiment set requires the same parameters to handle different joint counts, kinematic structures, body proportions and actuators. Training a separate controller for every robot–motion pair quickly becomes impractical.

Physics-based imitation with reinforcement learning (RL) produces complex skills from reference motion [1], and recent systems extend tracking to large motion sets and flexible motion commands [2], [3]. In parallel, embodiment-aware policies share parameters across robots by conditioning control on descriptions of the body [4], [5]. These approaches motivate a controller that combines varied whole-body motion tracking with parameter sharing across heterogeneous robots.

We propose a single embodiment-aware universal humanoid policy that combines these two directions by training jointly on several retargeted motions and humanoid robot topologies. Human motion-capture sequences are retargeted to each robot, and body parameters are randomized during training to expose the controller to a wider range of embodiments. The resulting policy uses the same parameters for every joint and robot, without robot-specific action heads. A central design choice is how the desired motion should be represented. A current retargeted joint target specifies where

each joint should be, but provides little information about how the motion is progressing. Short reference windows provide temporal context and desired-velocity information; their individual contributions are not isolated. We also investigate whether a compact quantized motion code produced by finite scalar quantization (FSQ) [6] can serve as a general motion command interface across embodiments, motivated by the recent success of token-based interfaces for multi-motion tracking [3].

This work makes three contributions. First, we extend a shared, descriptor-conditioned per-joint controller to motion tracking across five simulated humanoid models under body randomization. Second, we compare current-target, raw-window and quantized-window inputs, documenting an upper-body tracking instability in reference-only training under the tested curriculum. Third, we evaluate the FSQ representation through control without a direct joint-reference input, reduced command-update rates, held-out LAFAN1 [7] and DanceDB [8], [9] sequences, and exploratory held-out-robot and cross-body tests.

II. RELATED WORK

A. Physics-Based Imitation and Motion Tracking

DeepMimic demonstrated reinforcement learning for tracking reference poses while maintaining balance [1]. BeyondMimic adds a guided diffusion model that adapts motion plans to task constraints [2]. SONIC scales whole-body tracking in model size, data and compute, and uses a common token interface for different motion command sources, with motion encoders that use multiple future reference frames [3]. Its tracker is trained for one target robot. Temporal context is also used in multi-humanoid controllers: VENOM combines current reference motion with reference history and a future reference [10], while LocoGPT conditions autoregressive locomotion generation on a history of state observations [11].

Retargeting quality is also important: General Motion Retargeting studies its effect on downstream tracking [12], while ReActor adapts references for physical feasibility jointly with the policy [13]. For this work, retargeted trajectories are fixed inputs and are not a learned component.

B. Multi-Embodiment Policies

The Unified Robot Morphology Architecture (URMA) shares per-joint modules across robots through joint and robot descriptors [4]. URMAv2 extends this approach with stronger embodiment randomization and larger-scale locomotion training [5]; related work studies scaling with the

¹Department of Computer Science, Technical University of Darmstadt, Darmstadt, Germany. ²Project supervisor.

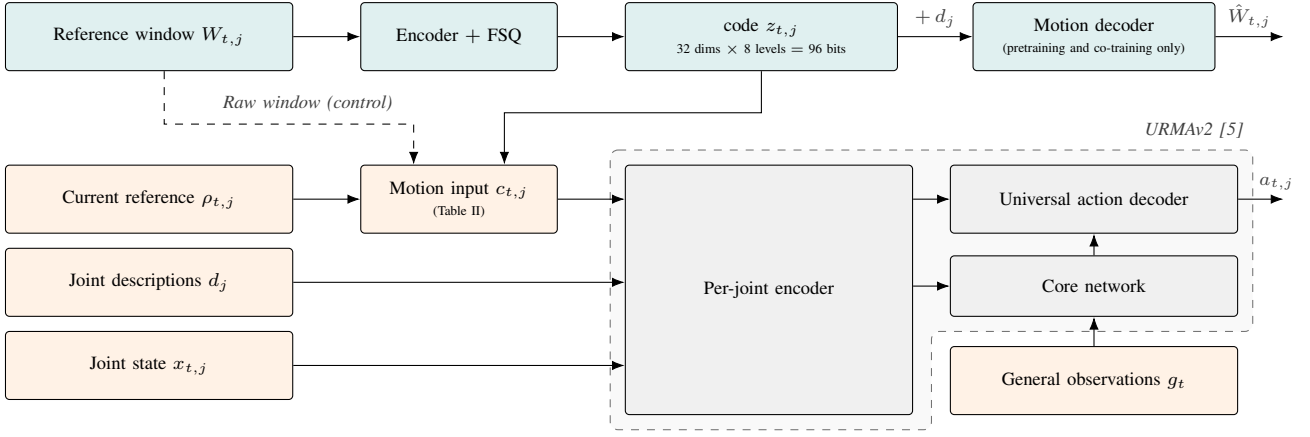


Fig. 1. Policy and motion interface. Each joint is processed using its descriptor, state and motion input $c_{t,j}$. Depending on the condition, $c_{t,j}$ contains the current reference, the FSQ code $z_{t,j}$, or both; the dashed path substitutes a raw window for the code. These are alternative inputs, not simultaneous branches. The decoder is used only for reconstruction during tokenizer training and co-training. The critic additionally receives exact simulated body properties and contact information.

TABLE I

DESIGN COMPARISON WITH RELATED SYSTEMS. “QUANTIZED MOTION CODE” DENOTES A DISCRETE REPRESENTATION USED TO CONDITION THE CONTROLLER; REPRESENTATION LEARNING IS ABBREVIATED AS REPR. LEARNING.

Method	Control task	Multi-Humanoid	Multi-Task	Robot-specific heads	Training	Quantized motion code
SONIC [3]	motion tracking	no	yes	–	RL + repr. learning	yes
LocoGPT [11]	locomotion	yes	yes	yes	supervised	no
VENOM [10]	motion tracking	yes	yes	yes	supervised	no
URMA, URMAv2 [4], [5]	velocity commands	yes	no	no	RL	no
This work	motion tracking	yes	yes	no	RL + repr. learning	yes

number of training embodiments [14]. These policies follow velocity commands instead of whole-body motion references.

VENOM is the closest comparison: a shared transformer tracks whole-body motion on several humanoids using supervised learning from robot-specific reinforcement-learning experts, with robot-specific input and output embeddings [10]. In contrast, our controller uses shared per-joint modules conditioned on embodiment descriptors and is trained directly by reinforcement learning. We combine this architecture with body randomization and study the choice of motion input. Table I provides an overview of the design choices across the related approaches. Our experiments focus on motion conditioning within a shared-policy framework.

III. PROBLEM FORMULATION

A. Task Definition

Let $\mathcal{E} = \{e_1, \dots, e_N\}$ denote the robots and $\mathcal{M} = \{m_1, \dots, m_K\}$ the source motions. Each training episode samples a robot e , a motion m and a randomized variant $\xi \sim p(\xi | e)$ of that robot, which changes body dimensions, masses, inertias, joint and actuator parameters within the robot’s family.

The robots have different joint spaces, so each source motion is retargeted separately:

$$\rho_{0:T_m}^{e,m} = \mathcal{R}_e(m), \quad (1)$$

where $\mathcal{R}_e$ maps motion m to the joint order, limits and root convention of robot e . Actuators without a counterpart in the motion stay at their nominal position. The same retargeting pipeline is used for training and evaluation; held-out motion sequences are excluded from training.

One policy is trained over all robots, motions and variants:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{e,m,\xi,\tau} \left[\sum_{t=0}^{T-1} \gamma^t r_{e,m,\xi}(s_t, a_t) \right]. \quad (2)$$

B. Shared Policy Interface

For every actuator $j \in \mathcal{J}_e$ of the current robot, the policy receives a joint state $x_{t,j}$, an embodiment descriptor $d_j^{e,\xi}$ and a motion input $c_{t,j}^{e,m}$. The descriptor encodes where the joint sits in the body, its axis and limits, the mass and inertia it moves, and its actuator properties. A global input $g_t^{e,\xi}$ carries the base state and robot-level properties. The policy is

$$a_t^e \sim \pi_{\theta} \left(\{d_j^{e,\xi}, x_{t,j}, c_{t,j}^{e,m}\}_{j \in \mathcal{J}_e}, g_t^{e,\xi} \right), \quad (3)$$

with one set of parameters for every robot. Robots with fewer actuators are padded to a common size, and a mask marks which slots are real.

The motion input $c_{t,j}$ denotes the complete motion conditioning, whereas $z_{t,j}$ denotes only the FSQ code. Thus, $c_{t,j} = \rho_{t,j}$ for reference-only control, $c_{t,j} = [\rho_{t,j}, z_{t,j}]$ for reference-plus-code control, and $c_{t,j} = z_{t,j}$ for token-only control. Raw-window variants replace the code with the

unquantized reference window. Selected input components can also be held between command updates. The reward still uses the current reference.

IV. METHOD

A. Embodiment-Aware Shared Policy

We extend URMAv2’s shared per-joint architecture [5] with reference-motion conditioning (Fig. 1). The same joint encoder processes each joint’s state, descriptor and motion input. Learned aggregation combines the joint features into a robot-level representation, which enters the core network together with the global input and is mapped back to one action per joint. There are no robot-specific layers. Padded joint slots are masked out of the actions and loss.

The actor receives noisy descriptors of link geometry, masses and inertias, joint and actuator properties, controller gains, and robot-level size and mass. The full descriptor inventory is given in Table V. The critic receives exact simulated body properties and privileged contact information, including contact timing and foot velocities. Action delays, external pushes and the environmental randomizations listed in Table V are not provided explicitly to either network. The controller therefore adapts using approximate body information; robustness to a completely unknown body is not assumed.

B. Motion Representation

The reference window $W_{t,j}$ contains the joint’s desired position and velocity at the current and ten following frames, together with both foot heights. The foot-height channels provide information about the motion beyond that joint alone. A per-joint encoder maps this window to an FSQ code $z_{t,j}$ with 32 dimensions, each quantized to eight levels. We use *token* and *FSQ code* interchangeably. The code either complements or replaces the direct current-reference input.

We compare two encoder-training strategies. In the *pre-trained, frozen* condition, the encoder and a descriptor-conditioned decoder are first trained to reconstruct windows from their codes; the encoder is then fixed during policy training. In *co-training*, the pretrained encoder and quantizer are placed inside the policy and updated using the reinforcement-learning objective together with a reconstruction loss. A straight-through estimator passes gradients through quantization during training [6].

Two ablations examine whether quantization or future reference frames are necessary for the observed improvement. The first supplies the raw window through the same motion-input path, without a learned motion encoder or quantizer. The second uses the current and ten preceding frames instead of future frames. Table II lists the conditions. Both raw and quantized windows contain desired velocities and foot heights that the current-target baseline lacks; these comparisons therefore do not isolate multiple temporal frames from richer instantaneous features. Reduced-update-rate experiments hold the selected input between updates while simulation and reward computation continue.

C. Randomization

Each training episode samples a body variant. A coefficient $c \in [0, 1]$ scales the width of the body-randomization ranges: $c = 0$ gives the nominal body, and $c = 1$ gives the full configured ranges. The randomized quantities are listed in Table VII. Increasing c during training first exposes the policy to nearly nominal bodies and then to larger variations. Observation noise, action delay and external pushes are applied throughout. The training schedule is specified in Table VI.

V. EXPERIMENTAL SETUP

A. Simulation, Robots and Motion Data

The system is built on RL-X [15] and the URMAv2 environment [5] in JAX, using MuJoCo [16] with MJX for parallel physics. The policy runs at 50 Hz and outputs joint position targets for a PD controller. Five humanoids are used: Unitree H1 and G1, Booster T1, Boston Dynamics Atlas and Appttronik Apollo, with 19 to 30 tracked actuators each. The controlled two-robot experiments use H1 and G1.

Motions come from LAFAN1 [7]. Twenty sequences (dances, walks, runs, a sprint, jumps and fights) form the training set; seven additional sequences listed in Table IV are held out and never used for policy training or tokenizer fitting. Six external motion-capture sequences (dances) from DanceDB [8] via AMASS [9] form a second held-out set. Every motion is retargeted to every robot with the same pipeline. Joint trajectories beyond a robot’s range are clamped. Episodes start from a random frame of the motion and end when the robot falls, detected as the pelvis dropping below a fraction of its nominal height. We do not terminate episodes solely on tracking deviation. Falls can nevertheless shorten episodes, so a common maximum duration does not imply equal evaluated trajectory lengths; pre-fall errors must be interpreted separately from full-horizon tracking.

B. Policy, Reward and Training

The reward combines exponential kernels for joint-position and joint-velocity tracking, foot height and swing timing, and root heading. Penalties discourage abrupt actions, foot slip and ground penetration, alongside the gait regularizers of URMAv2. Root heading is included in the observation. Policies are trained with proximal policy optimization (PPO) [17]. Network widths, reward weights and optimizer settings are given in Table VI. Each policy update uses an equal number of samples from every robot.

Training compute is reported in budget units: $1 \times$ corresponds to 19.7M environment steps on H1 and G1, with larger budgets multiplying this count. Four- and five-robot policies train for 59M steps, taking approximately four hours on one 16GB GPU. The standard randomization schedule increases c from 0.2 to 0.7 over 40M steps. The capped-schedule ablation is reported in Fig. 9. Consequently, shorter runs stop during the ramp, whereas longer runs include training at its final level. Policies with $c = 0$ are trained on the *nominal body* without body randomization.

C. Motion Tokenizer

The tokenizer is pretrained on the H1 and G1 training motions. Frozen encoders are then applied to other robot retargets without additional fitting, while co-trained encoders continue to update during policy training. We use a mean-pool encoder, which pools the window over time, and a flattened-window encoder, which retains its temporal arrangement before quantization. The main reference-plus-token experiments use the former, and token-only control uses the latter. Encoder ablations and reconstruction errors are reported in Fig. 16 and Table XI. In the policy, the code enters through a small per-joint projection alongside the joint state.

D. Evaluation Protocol

We evaluate frozen policies in rollouts starting from random motion phases, using 64 parallel environments for two-robot policies and 16 for multi-robot policies, with a maximum episode duration of 20 s. Unless stated otherwise, robot bodies are nominal and noise and external pushes are disabled. Results aggregate these rollouts for the robot and motion sets indicated in each figure or table. When the tested robot, motion or robot–motion pair was excluded from policy training, we refer to the evaluation as *zero-shot*. Here, zero-shot means that the policy is applied without further training, not that the robot model, embodiment descriptors or retargeted references are unavailable.

Tracking accuracy is measured using *demeaned joint-angle RMSE* in radians. Each executed and reference joint trajectory is centred before comparison, so a constant angular offset is not counted as tracking error. We report this metric over all tracked joints and separately over arm and leg joints, with root-heading error where relevant. To put the tracking errors in context, we compare policies against a *zero-action baseline* under the same evaluation conditions. This baseline replaces policy outputs with nominal PD targets; it therefore uses static position targets rather than zero joint torques. Tables IX–X report root-relative link errors, with success rates in Table IX and pre-fall Apollo errors in Table X.

We also evaluate how tracking changes when motion commands are updated less frequently. These command-update experiments use a 40 Hz motion-reference stream, with selected inputs refreshed at 40, 8, 4 or 2 Hz and held constant between updates, while the controller continues to run at 50 Hz. This changes how often the policy receives updated motion information without changing its control rate.

Across the evaluations, reported means and standard deviations summarize independent training seeds where multiple seeds are available. The reported errors also vary with the evaluation seed: repeating an evaluation with a different seed produced changes of up to 5 %. We therefore interpret differences of this scale cautiously and not as consistent improvements.

TABLE II

MOTION-INPUT CONDITIONS. THE CONTROLLER AND REWARD ARE SHARED; COMPARISONS USE MATCHED ROBOT AND MOTION SETS AND TRAINING BUDGETS WHERE AVAILABLE. “REFERENCE” MEANS THE CURRENT DESIRED JOINT POSITION, AND “CODE” MEANS THE FSQ REPRESENTATION OF A MOTION WINDOW. BUDGET COVERAGE IS LISTED IN TABLE VIII.

Condition	Motion input per joint	Encoder
reference only	current reference	–
+ token (precomputed)	reference + code	frozen
+ token (co-trained)	reference + code	in the policy
+ raw window (future)	reference + window	none
+ raw window (past)	reference + past window	none
token only	code only	frozen

TABLE III

FIVE-ROBOT POLICIES EVALUATED ON NOMINAL BODIES: DEMEANED JOINT RMSE IN RADIANS ON TRAINING AND HELD-OUT LAFAN1 MOTIONS. THE ZERO-ACTION BASELINE USES NOMINAL PD TARGETS. LOWER IS BETTER; THE LOWEST VALUE WITHIN EACH ROBOT AND MOTION-SET GROUP IS BOLD. BOLDFACE INDICATES NUMERICAL RANK, NOT STATISTICAL SIGNIFICANCE.

Robot	training motions			held-out motions		
	baseline	ref. only	+ token	baseline	ref. only	+ token
H1	0.383	0.392	0.189	0.343	0.366	0.191
G1	0.373	0.231	0.161	0.312	0.226	0.158
T1	0.362	0.360	0.183	0.327	0.324	0.178
Atlas	0.500	0.240	0.231	0.468	0.237	0.218
Apollo	0.324	0.247	0.195	0.297	0.227	0.157

VI. RESULTS

We first evaluate tracking across humanoids and held-out motions (Secs. VI-A and VI-B), then examine training stability (Sec. VI-C) and the quantized command interface (Sec. VI-D). Unless stated otherwise, reported errors are for H1 on the training motions at the full 40 Hz command-update rate.

A. One Policy Across Five Humanoids

At the same 59 M-step budget, the five-robot reference-plus-token policy has approximately 4–52 % lower mean demeaned joint RMSE than the reference-only policy across the five trained robots (Fig. 2a, Table III). Relative to nominal PD targets, the reduction is 40–57 %. The reference-only policy uses the same training budget, randomization and motions, but reaches approximately baseline error on H1 and T1; its error is closer to the token policy on Atlas. The four-robot comparison shows the same broad pattern. Replacing the FSQ code with a raw reference window also enables tracking below the baseline on all four trained bodies. Section VI-C examines how the reference-only degradation develops during training.

The four-robot token-only policy also tracks all trained bodies below the baseline, without receiving direct joint targets. On held-out Apollo, only the reference-plus-FSQ policy improves over the baseline, by approximately 10–15 %. This

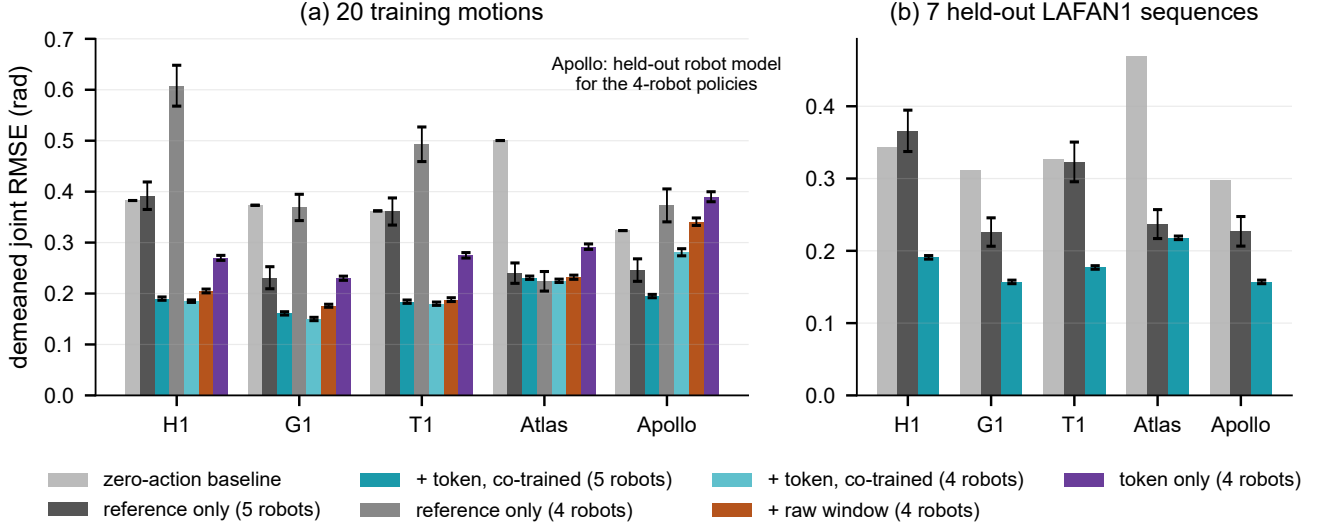


Fig. 2. Multi-robot policies trained for 59M steps with body randomization and evaluated on nominal bodies; error bars show standard deviations across three training seeds where available. (a) Demeaned joint RMSE on the 20 training motions. Four-robot policies are trained on H1, G1, T1 and Atlas; Apollo is held out. (b) Five-robot policies on the seven held-out LAFAN1 sequences. Window-conditioned policies improve over the baseline on all trained bodies; this does not imply successful transfer to every held-out body.

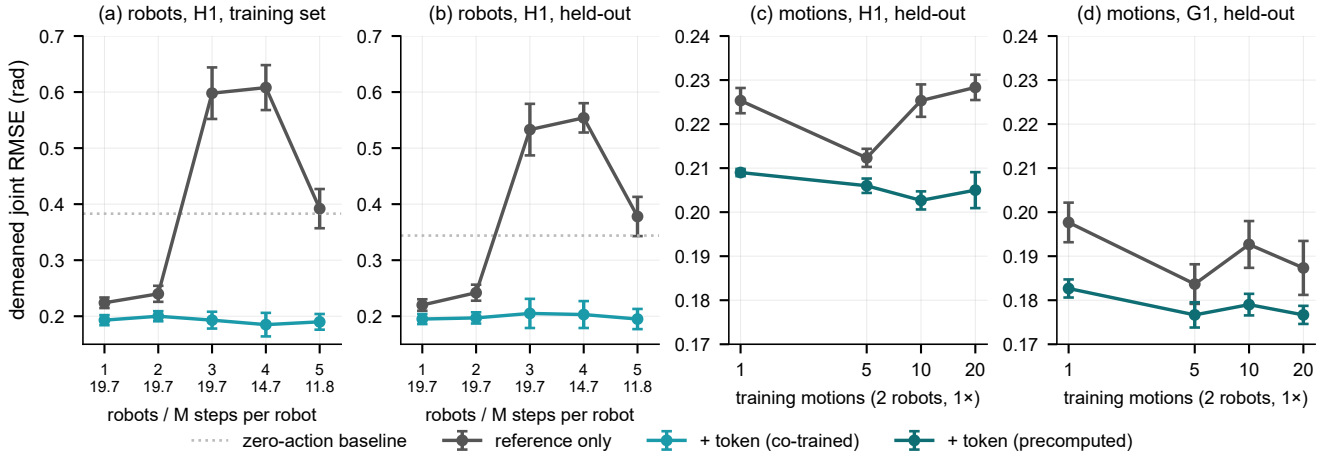


Fig. 3. Scaling the number of robots and training motions. (a, b) H1 demeaned joint RMSE on training and held-out LAFAN1 sequences for policies trained on one to five humanoids. The second tick line gives millions of training steps per robot; total budgets are 19.7M for one robot, 39.3M for two, and 59M for three to five. (c, d) Held-out error for two-robot policies trained on 1, 5, 10 or 20 motions at the $1\times$ budget. Error bars show standard deviations across three training seeds where available. The reference-plus-token policy retains similar H1 error as robots are added. Increasing the motion count does not consistently reduce held-out error in these experiments.

result does not demonstrate transfer through the code alone, since the policy still receives Apollo’s current joint target and embodiment descriptor. Moreover, the additional link-space evaluation reports falls on held-out Apollo (Table X), so lower tracking error does not establish reliable full-episode control.

As the number of training robots increases, the reference-plus-token policy maintains similar H1 error despite fewer training steps per robot (Fig. 3a,b). The shared five-robot policy matches or improves on the corresponding single-robot token policy on four of the five robots (Fig. 13). In this scaling experiment, reference-only H1 error is markedly

higher with three or more robots. The longer two-robot sweeps below show that the outcome also depends on training duration, not robot count alone.

Selected simulation rollouts and motion-input comparisons are provided in the supplementary videos. These examples complement the quantitative evaluation.

B. Held-Out Motion Sequences

On the seven held-out LAFAN1 sequences, the five-robot reference-plus-token policy remains 44–53 % below the baseline, with errors comparable to its training-motion aggregate (Fig. 2b, Table III). On the six external DanceDB sequences,

its H1 and G1 errors are 0.19 and 0.16rad, respectively. The reference-only five-robot policy retains useful held-out tracking primarily on the robots where it also tracks the training motions.

The two-robot experiments show a similar aggregate advantage for richer motion inputs (Fig. 15). Compared with the current target alone, window conditioning reduces error by approximately 9–11 % on held-out LAFAN1 sequences, versus about 5 % on training motions. Reference-plus-token and raw-window policies also reduce aggregate tracking error on DanceDB. Token-only control achieves comparable tracking error to reference-only control on held-out LAFAN1 sequences, but approximately 14 % higher error on DanceDB. Because the DanceDB retargets are clamped, these errors describe tracking of the processed robot references and do not directly reflect fidelity to the original human motion.

Increasing the training set from one to 20 motions does not consistently lower held-out error at the tested budget (Fig. 3c,d). Even policies trained on one motion respond to held-out references, but this result alone does not establish how the policy represents or generalizes motion. The five-robot reference-plus-token policy remains below the baseline for every evaluated held-out robot-sequence combination (Fig. 12).

C. Training Stability Under Body Randomization

Observed Training Instability. In the four-robot example, reference-only arm tracking is initially comparable to the other conditions but deteriorates as training continues under the randomization schedule (Fig. 5). The corresponding token, raw-window and token-only runs do not show this sustained degradation. Per-joint correlations localize the change to several arm joints, particularly elbows and shoulder rotation, while the rest of the body remains comparatively unaffected (Fig. 14). The affected joints continue to move but no longer closely follow their references.

Training Budget. The replicated H1–G1 sweep shows that reference-only tracking can deteriorate before recovering with substantially more training (Fig. 4). Arm error rises at $2\times$ and $4\times$, partially recovers at $8\times$, and falls below its $1\times$ level at $16\times$. At $16\times$, all-joint error is comparable to that of quantized-window policies trained for half as many steps. Thus, the observed failure is not a permanent representational limit. Nominal-body training does not show the same intermediate-budget deterioration (Fig. 9). A short additional training phase after degradation does not restore tracking in the tested runs (Fig. 8).

Stabilization. At matched budgets, a raw future window achieves accuracy comparable to the FSQ code without a learned motion encoder, quantization or reconstruction loss (Figs. 4 and 7). A raw past window provides a similar benefit across the full tested budget range. Quantization and an explicit future window are therefore not necessary for these observed gains. However, the windows also add desired velocities and foot heights. The experiments do not determine whether multiple temporal frames are necessary, or whether simpler direction or motion-phase cues would suffice.

Co-training the encoder produces no consistent accuracy advantage over a frozen pretrained encoder under the tested objectives and budgets. Encoder architecture and projection width likewise have little effect when the direct reference is present (Fig. 16). Removing the code at evaluation worsens tracking without reproducing the severe training-time failure. This indicates that the policy uses the code at run time, while the training comparisons associate richer motion conditioning with improved stability.

Upper- and Lower-Body Tracking. The largest differences between motion-input conditions occur in the arm joints. Leg-joint errors are more similar across conditions and improve mainly with training budget (Fig. 4). Tripling the leg-tracking reward weight did not appreciably improve them in the tested ablation. Under randomized-body evaluation, window-conditioned policies generally lose accuracy as the variation increases, while retaining their broad advantage over reference-only control (Figs. 10 and 11).

D. Properties of the Quantized Interface

Raw and quantized windows achieve comparable tracking accuracy over matched budgets. We therefore evaluate the FSQ code separately as a compact command representation: whether it can replace direct joint targets, tolerate less frequent updates and be reused across bodies. These properties concern the tested quantized interface and do not establish that quantization is uniquely necessary for each capability.

Control without a Direct Joint Reference. With the flattened-window tokenizer, the token-only policy tracks the training and held-out motions below the static-target baseline. Its held-out LAFAN1 error is comparable to reference-only control, although DanceDB tracking is less accurate (Fig. 15). This establishes that the low-level policy can use a discrete per-joint command stream without a separate current-target input. The commands still encode retargeted reference motion; no reference-free motion generation is demonstrated. The mean-pool tokenizer did not support successful token-only training in the tested configuration. The corresponding reconstruction errors are reported in Table XI.

Reduced Command-Update Rates. Lower update rates generally increase error (Fig. 6). Token-only control remains below the baseline at 2 Hz. When a fresh direct reference is retained, holding the code causes somewhat less degradation than holding the raw window, but this is not a reduction in the update rate of the entire motion input. Except for the explicitly labelled 2 Hz-trained variants, policies are trained with fresh inputs and receive no input-age signal. The experiment therefore tests tolerance to stale commands rather than a controller designed for asynchronous operation.

Held-Out-Robot and Cross-Body Tests. The Apollo result in Sec. VI-A shows a tracking-error benefit from additional FSQ conditioning, not reliable transfer through the code alone. A separate test drives the token-only policy with codes computed from another robot’s retarget of the same walk (Fig. 18). The most favourable transfers occur among H1, G1 and T1 but they do not extend consistently to Atlas and Apollo. Since this test covers one motion, the

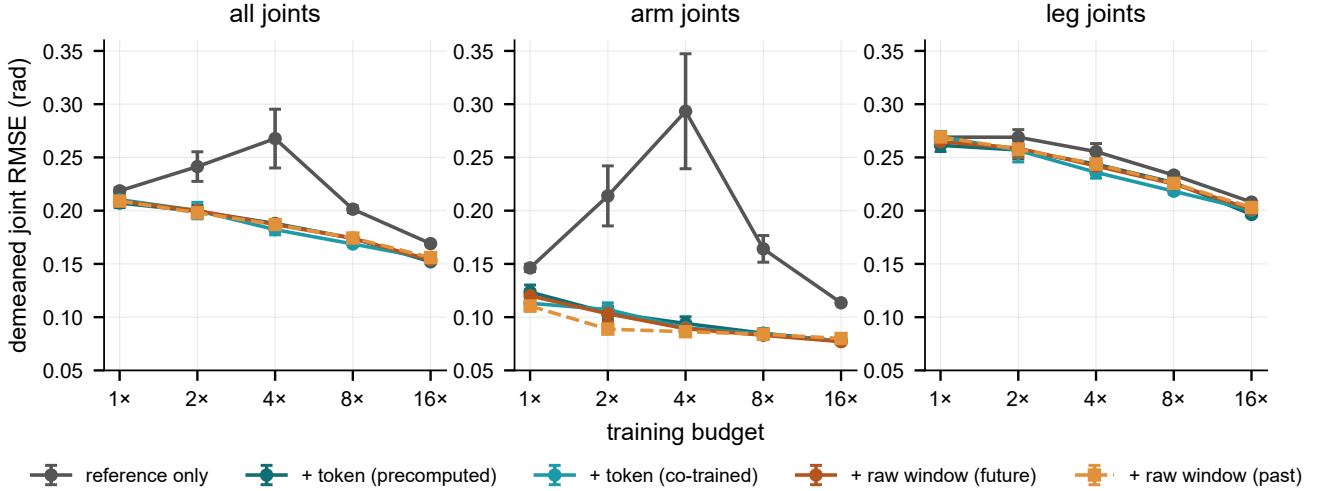


Fig. 4. Training-budget sweep for two-robot policies, evaluated on nominal H1 bodies. Points show mean demeaned joint RMSE and error bars show standard deviations across training seeds; counts are in Table VIII. The budgets span 19.7 M (1 $\times$) to 315 M (16 $\times$) environment steps. Reference-only upper-body tracking degrades at intermediate budgets and recovers with further training. Quantized-window policies improve throughout the sweep. Raw future and past windows achieve accuracy comparable to quantized windows across all tested budgets. Lower-body differences are smaller.

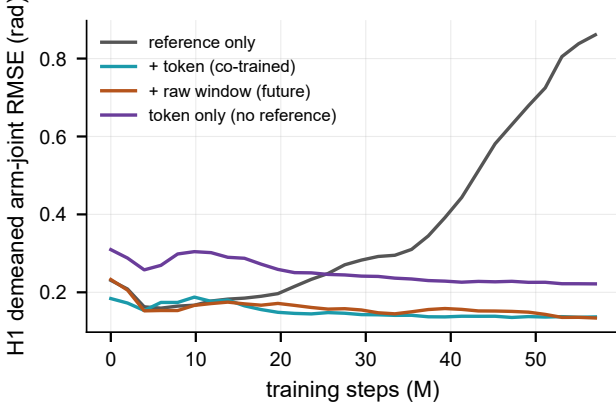


Fig. 5. H1 arm-joint error during four-robot training with body randomization. Each curve is one training run, evaluated on the nominal body at successive checkpoints. The reference-only policy initially tracks the arms but degrades later; the reference-plus-token, raw-window and token-only variants do not exhibit the same sustained increase in error.

results support limited code reuse, while evidence for an embodiment-invariant motion representation remains limited.

Representation Size. A 32-dimensional code with eight levels per dimension requires 96 bits per joint and update if bit-packed. A raw 11-frame window with four 32-bit values per frame requires 1408 bits in its per-joint form. These are nominal representation sizes, not measured communication payloads. Reference-plus-token conditions also include the current joint target, and protocol overhead is excluded.

E. Tracking Limitations

Lower-body tracking remains the main weakness: leg-joint errors are much closer to the zero-action baseline than arm-joint errors, and increased leg-reward weighting did not resolve this. Contact dynamics and partly infeasible

retargets are possible contributors, but their effects are not isolated. Root heading is also imperfect: multi-robot policies trained with body randomization can drift by tens of degrees. Increasing the heading-reward weight reduces drift at the expense of joint-tracking accuracy (Fig. 17).

Demeaned joint RMSE excludes constant angular offsets, while root-relative link-position error excludes global translation. In the single-seed H1-G1 link-space evaluation, globally aligned MPJPE was 2–8 m over the 20 s rollout, dominated by root drift (Table IX). Improvements in local pose tracking therefore do not establish accurate global trajectory following.

VII. DISCUSSION

A. Interpretation

The results distinguish the role of richer motion conditioning from the choice to quantize it. Under the tested randomization curriculum and optimization settings, reference-only policies sometimes develop a long-lived upper-body tracking failure. None of the richer-input variants shows the same behaviour over its evaluated budgets. Raw and quantized windows perform similarly, so the improvement does not require quantization. The mechanism remains unresolved because the windows add desired velocities and foot heights as well as multiple frames.

We observe a non-monotonic interaction between training duration and the tested body-randomization curriculum. Capping its maximum level did not remove the intermediate-budget degradation (Fig. 9). This suggests that the final randomization level alone does not explain the result; it does not establish a general rule that longer randomized training is destabilizing.

The controller uses noisy descriptors of the randomized body. The results therefore demonstrate adaptation using

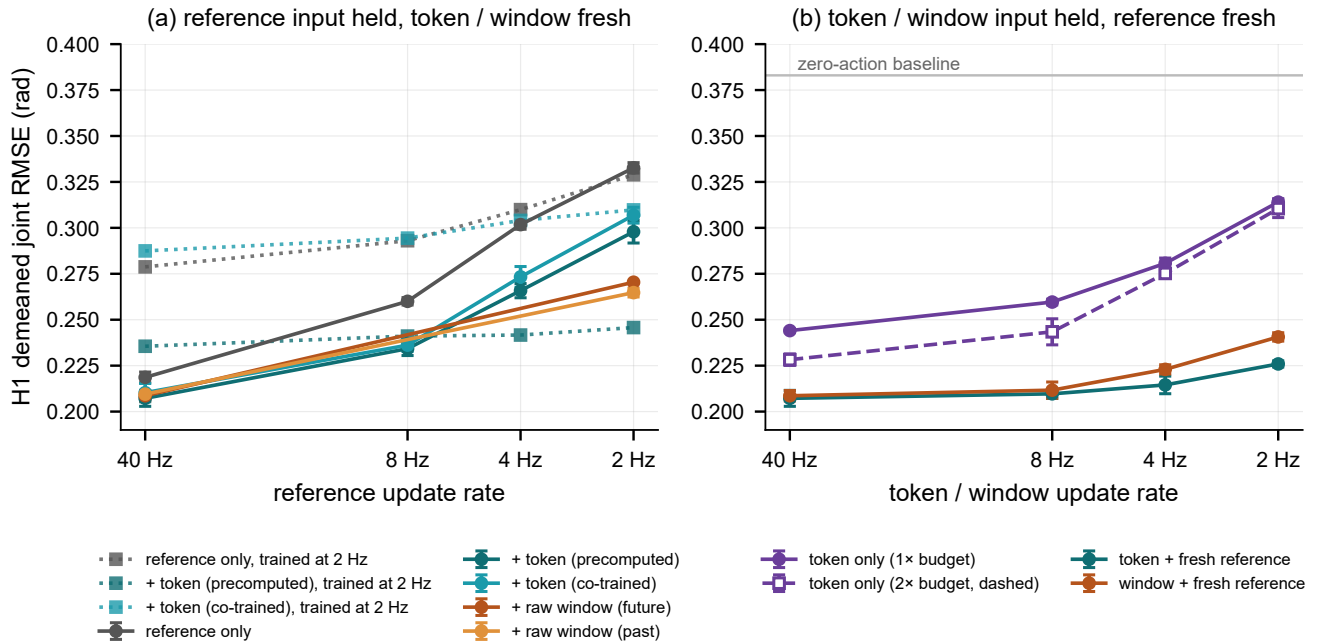


Fig. 6. Reduced command-update rates for two-robot policies on H1; means and standard deviations across three training seeds are shown. (a) The current-reference input is held between updates while any token or window input remains fresh. Dotted curves are policies trained with 2 Hz reference updates. (b) The token or window is held, while the direct reference remains fresh where present. In panel (b), only the token-only conditions reduce the update rate of the complete motion input. All token-only curves remain below the zero-action baseline at 2 Hz.

approximate embodiment information within the tested distribution, rather than control without body knowledge. The Apollo result likewise retains a robot-specific reference and descriptor, and its lower tracking error should be considered alongside the falls reported in the additional evaluation.

Our encoder co-training experiment tests reinforcement-learning gradients through FSQ together with reconstruction. It does not reproduce SONIC’s full multi-source alignment and cycle-consistency objectives [3]; the absence of a consistent co-training gain here is therefore specific to our objective and budgets. Relative to VENOM [10], the controller avoids robot-specific layers and expert trajectories, but is evaluated on a smaller motion set and does not establish comparable overall tracking performance.

B. Limitations

All evidence is from simulation, with no hardware validation. A new robot still requires a model, descriptors and retargeted references. Multi-robot experiments have limited training-seed replication, and cross-body code reuse is tested on one motion; the replicated two-robot sweeps provide the stronger evidence about training behaviour. Demeaned RMSE excludes constant pose offsets, and fall-shortened trajectories require separate interpretation. The clamped DanceDB references and reduced-update tests further limit conclusions about human-motion fidelity and asynchronous deployment. Evaluation covers five humanoid models and their randomized variants; transfer to substantially different morphologies remains untested.

C. Future Work

A matched single-frame baseline with desired velocity and foot heights would help distinguish richer instantaneous features from multi-frame context. Further work should extend cross-body tests to more motions, examine contact dynamics and physically feasible lower-body references, and train explicitly for reduced-rate commands with input-age information. Hardware evaluation would then test whether the observed benefits extend beyond simulation.

VIII. CONCLUSION

We presented a shared, descriptor-conditioned controller for retargeted motion tracking across five simulated humanoid models. Richer motion inputs avoid an observed upper-body training instability over the tested budgets, while raw and quantized windows achieve comparable accuracy. This supports richer motion conditioning without isolating multiple frames from desired-velocity and foot-height cues. The FSQ representation provides a compact command interface that can replace the direct joint-reference input and remain usable at reduced update rates. Held-out-motion results demonstrate the approach’s scope, while lower-body tracking, heading accuracy and reliable cross-body transfer remain unresolved.

REFERENCES

- [1] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, “DeepMimic: Example-guided deep reinforcement learning of physics-based character skills,” *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 143:1–143:14, 2018.

- [2] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, “BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion,” 2025, arXiv:2508.08241.
- [3] Z. Luo, Y. Yuan, T. Wang, *et al.*, “SONIC: Supersizing motion tracking for natural humanoid whole-body control,” *Science Robotics*, vol. 11, no. 117, p. ead4592, 2026.
- [4] N. Bohlinger, G. Czechmanowski, M. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, “One policy to run them all: An end-to-end learning approach to multi-embodiment locomotion,” in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, 2025.
- [5] N. Bohlinger and J. Peters, “Multi-embodiment locomotion at scale with extreme embodiment randomization,” 2025, arXiv:2509.02815.
- [6] F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen, “Finite scalar quantization: VQ-VAE made simple,” in *International Conference on Learning Representations*, 2024.
- [7] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal, “Robust motion in-betweening,” *ACM Transactions on Graphics*, vol. 39, no. 4, pp. 60:1–60:12, 2020.
- [8] A. Aristidou, A. Shamir, and Y. Chrysanthou, “Digital dance ethnography: Organizing large dance collections,” *ACM Journal on Computing and Cultural Heritage*, vol. 12, no. 4, pp. 29:1–29:27, 2019.
- [9] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black, “AMASS: Archive of motion capture as surface shapes,” in *IEEE/CVF International Conference on Computer Vision*, 2019, pp. 5442–5451.
- [10] S. Padmanabhan, K. Miyazawa, and T. Horii, “Venom: Versatile embodied network for omni-bodied motion tracking,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.16696>
- [11] —, “LocoGPT: GPT-based multi-humanoid-task policy for humanoid locomotion,” *IEEE Access*, vol. 14, pp. 20905–20915, 2026.
- [12] J. P. Araújo, Y. Ze, P. Xu, J. Wu, and C. K. Liu, “Retargeting matters: General motion retargeting for humanoid motion tracking,” 2025, arXiv:2510.02252.
- [13] D. Müller, A. Serifi, S. Christen, R. Grandia, E. Knoop, and M. Bächer, “ReActor: Reinforcement learning for physics-aware motion retargeting,” *ACM Transactions on Graphics*, vol. 45, no. 4, pp. 97:1–97:11, 2026.
- [14] B. Ai, L. Dai, N. Bohlinger, D. Li, T. Mu, Z. Wu, K. Fay, H. I. Christensen, J. Peters, and H. Su, “Towards embodiment scaling laws in robot locomotion,” in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, 2025.
- [15] N. Bohlinger and K. Dorer, “RL-X: A deep reinforcement learning library (not only) for RoboCup,” in *RoboCup 2023: Robot World Cup XXVI*, ser. Lecture Notes in Computer Science. Springer, 2024.
- [16] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017, arXiv:1707.06347.

APPENDIX I DATA, OBSERVATIONS AND HYPERPARAMETERS

TABLE IV

MOTION AND ROBOT SETS. LAFANI SEQUENCE NAMES WITH THE SUBJECT IN PARENTHESES; THE SEVEN HELD-OUT SEQUENCES INCLUDE TWO WALKING SEQUENCES.

Set	Content
Train (20)	dance1 (1, 2), dance2 (1–5), walk1 (1, 2), walk2 (1), walk3 (1, 2), walk4 (1), run1 (2), run2 (1), sprint1 (2), jumps1 (1, 2), fight1 (2), fightAndSports1 (1)
Held-out (7)	dance1 (3), walk1 (5), walk3 (5), run2 (4), sprint1 (4), jumps1 (5), fight1 (5)
DanceDB (6)	Bachata, Bachata v2, Zeibekiko fast, Zeibekiko slow, Hasaposerviko, Zaloggo (two performers)
Robots	H1 (19 tracked joints), G1 (23 of 29; waist roll/pitch and wrists untracked), T1 (23), Atlas (27), Apollo (30)

TABLE V

OBSERVATIONS OF ACTOR AND CRITIC REGARDING THE RANDOMIZED BODY. “NOISY” MEANS THE VALUE IS PERTURBED BY MULTIPLICATIVE NOISE BEFORE IT ENTERS THE DESCRIPTOR.

Who	Quantities
Actor (noisy)	link scale; per-link mass, centre of mass and inertia; joint limits, damping, armature, friction, stiffness; torque and velocity limits; nominal joint position; PD and force gains; total mass; robot dimensions; actuator-dropout state
Critic (exact)	the same quantities at their exact simulated values; foot descriptors, foot contact states, time in contact and in the air, foot velocities; the current randomization level
Hidden from both	action delay, observation noise, external pushes, friction, gravity, contact-softness and fluid-density randomization
Both, per step	joint positions and velocities, base angular velocity, gravity direction, root heading, the motion input

APPENDIX II FULL TWO-ROBOT RESULTS

Table VIII gives the numbers behind Fig. 4. G1 values follow the same ordering at lower absolute error. Root-heading error at $1\times$ on H1: reference only 33° , precomputed token 25° , co-trained token 32° , raw window 16° , past window 24° , token only 14° . Reference-only policies trained without body randomization reach $0.232 / 0.231 / 0.223 / 0.174$ at $1\times / 2\times / 4\times / 8\times$. With the direct reference held at 2Hz, H1 error is 0.333 (reference only), 0.298 (precomputed token), 0.307 (co-trained token), 0.270 (raw future window) and 0.263 (raw past window); any additional token or window input remains fresh. Token-only policies have no direct-reference channel. Their reduced-rate results are reported separately in Fig. 6b.

Tables IX and X report the same policies in link space: the mean position error of the robot’s links after forward kinematics of the executed and the reference joint angles, relative to the pelvis.

TABLE VI

TRAINING AND TOKENIZER SETTINGS SHARED BY EVERY POLICY.

Setting	Value
Control / physics	50 Hz PD targets / 5 ms step, MJX
Rollout, epochs, minibatch	64 steps, 5, 8192 (two robots), $1024\times$ robots
Clip, γ , GAE λ , entropy	0.1, 0.99, 0.7, 0.03
Learning rate	Muon $1.5\cdot 10^{-2} \rightarrow 3\cdot 10^{-4}$ (core), Adam $1.5\cdot 10^{-3} \rightarrow 3\cdot 10^{-4}$ (encoders), linear
Max grad norm, KL stop	3.0, 0.15
Policy widths	joint encoder 256–256; state proj. 8; token proj. 4; core 512, 256, 256, 256, 256
Joint tracking kernel	$30 \exp(-\tilde{e}^2/0.05)$; heading ratio 0.2 (temp. 2.0), 0.5 in heading-fixed policies
Foot height / swing ratio	1/3 (temp. 0.05) / 1/2
Penalties	action rate 3.0, smoothness 0.1, foot slip 6.7, foot z -vel. 1.0, penetration 1000
Contact	solref time constant 4 ms
Termination	pelvis below 80 % of nominal height
Budget $1\times$	19.66 M steps, 768 envs, H1+G1
Multi-robot policies	59 M steps, 40–64 envs/robot, c : $0.2 \rightarrow 0.7$ over 40 M steps
Tokenizer window	11 frames $\times$ (pos., vel., 2 foot heights) per joint
FSQ code	32 dims $\times$ 8 levels = 96 bits; obs. divisor 10
Co-training	encoder init. from tokenizer; recon. weight 1.0; straight-through

TABLE VII

RANDOMIZED QUANTITIES. BODY PROPERTIES ARE SAMPLED PER EPISODE WITH RANGES SCALED BY THE COEFFICIENT c ; OBSERVATION NOISE, ACTION DELAY AND PUSHES FOLLOW THE URMAV2 DEFAULTS [5].

Group	Randomized quantities
Geometry ($\propto c$)	link scale (masses $\propto s^3$, inertias rescaled), foot size, joint axis tilt (0.1 rad), joint ranges, body and IMU positions, centre-of-mass offset
Dynamics ($\propto c$)	body masses and inertias (coupled and decoupled factors), inertia orientation, joint damping, armature, friction loss, stiffness
Actuation ($\propto c$)	PD gains ($\pm 50\%$), force gain, torque limits, nominal joint offsets, joint dropout
Observation	joint position 0.03 rad, joint velocity 1.5 rad/s, base velocity 0.2, gyro 0.2, gravity 0.05
Timing / external	action delay 0–20 ms; velocity and joint perturbations; friction, gravity, density and viscosity factors
Evaluation	nominal body, no noise, no pushes (or fixed c where stated)

APPENDIX III ADDITIONAL RESULTS

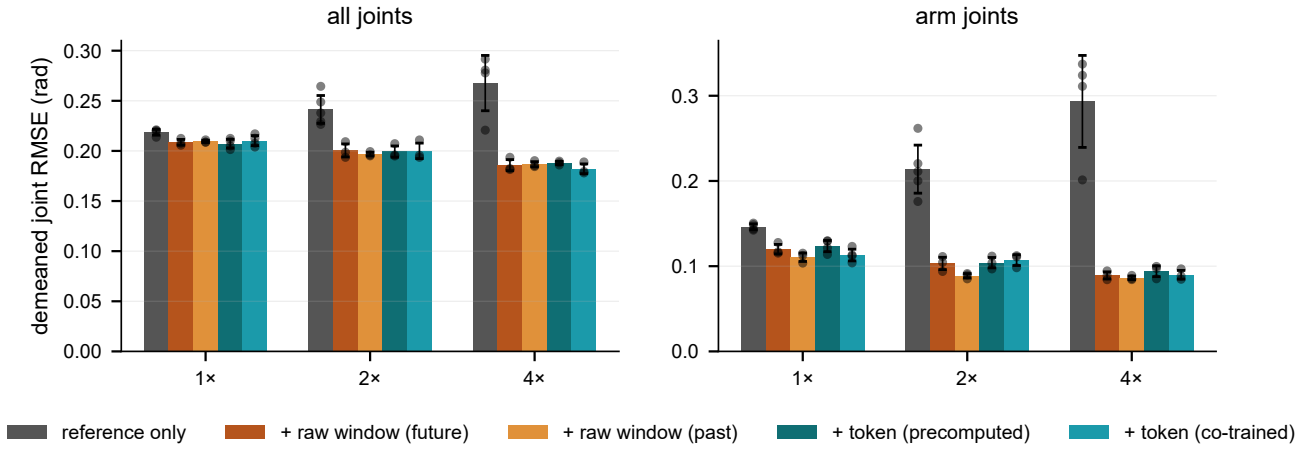


Fig. 7. Per-seed comparison of motion-input conditions on H1 at the full command rate. Dots represent independent training seeds. At matched tested budgets, raw future and past windows achieve accuracy comparable to the quantized inputs. Budget coverage is listed in Table VIII.

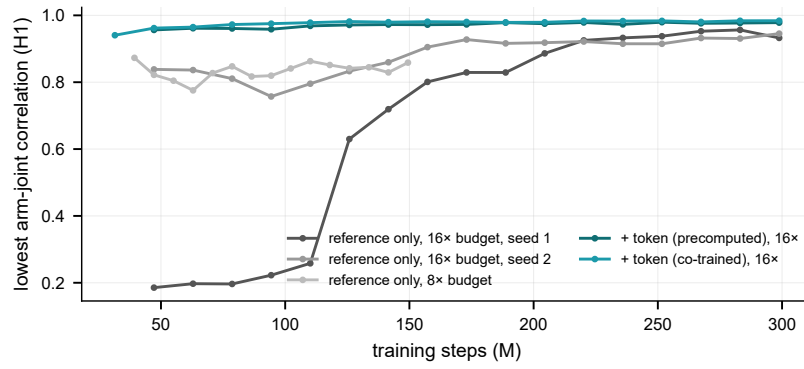


Fig. 8. Lowest per-joint correlation among the H1 arm joints during 16 $\times$ training (one training run per curve, evaluated at every eighth checkpoint). One reference-only seed loses an arm joint before 50M steps and recovers it only after about 150M; the token policies never drop. A degraded reference-only policy also does not recover within a 4M-step additional training phase on nominal or on randomized bodies (not shown).

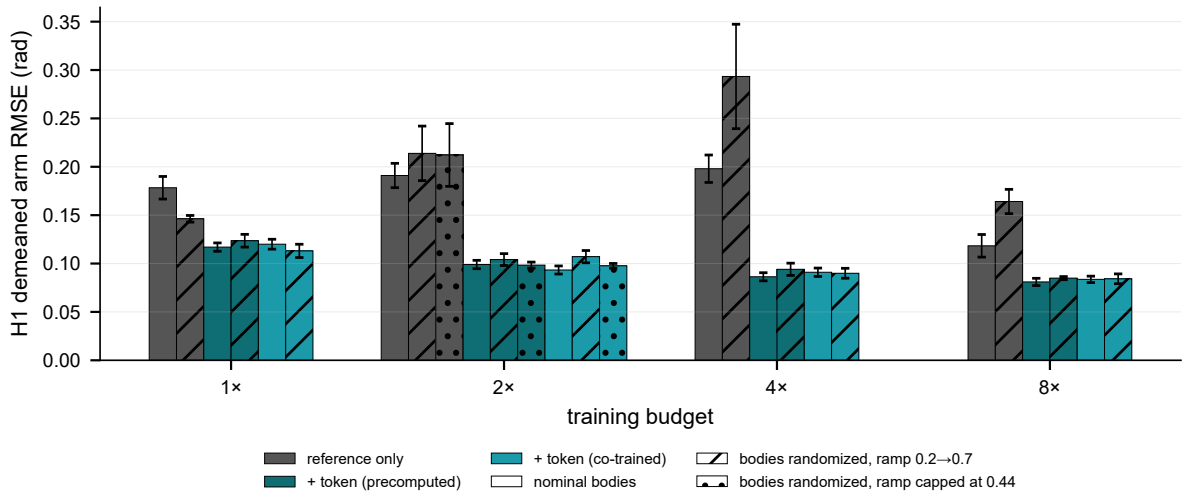


Fig. 9. Training-body distribution against budget (H1 arm-joint error, nominal-body evaluation, mean $\pm$ std over training seeds). Body randomization helps the reference-only policy at 1 $\times$ and hurts it beyond; capping the randomization does not remove the degradation. Token-conditioned policies remain comparatively stable across the tested schedules.

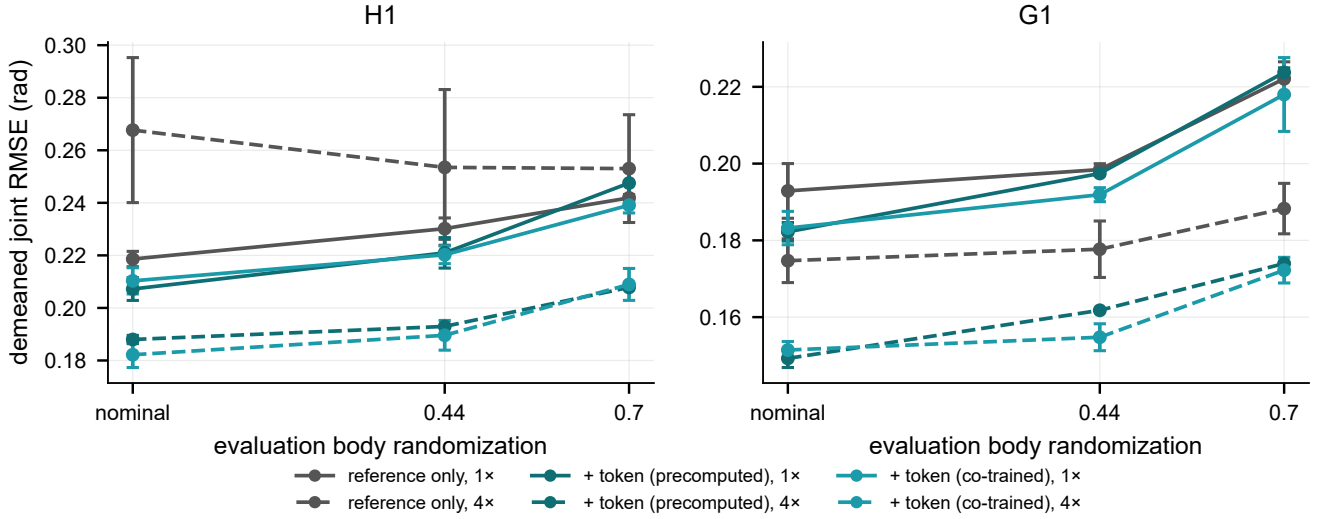


Fig. 10. Evaluation on randomized bodies (two-robot policies, mean $\pm$ std over training seeds). The 1 $\times$ policies stop during the randomization ramp, whereas the 4 $\times$ policies reach $c = 0.7$. Token-conditioned policies generally have higher RMSE as evaluation randomization increases; relative rankings are not preserved in every case.

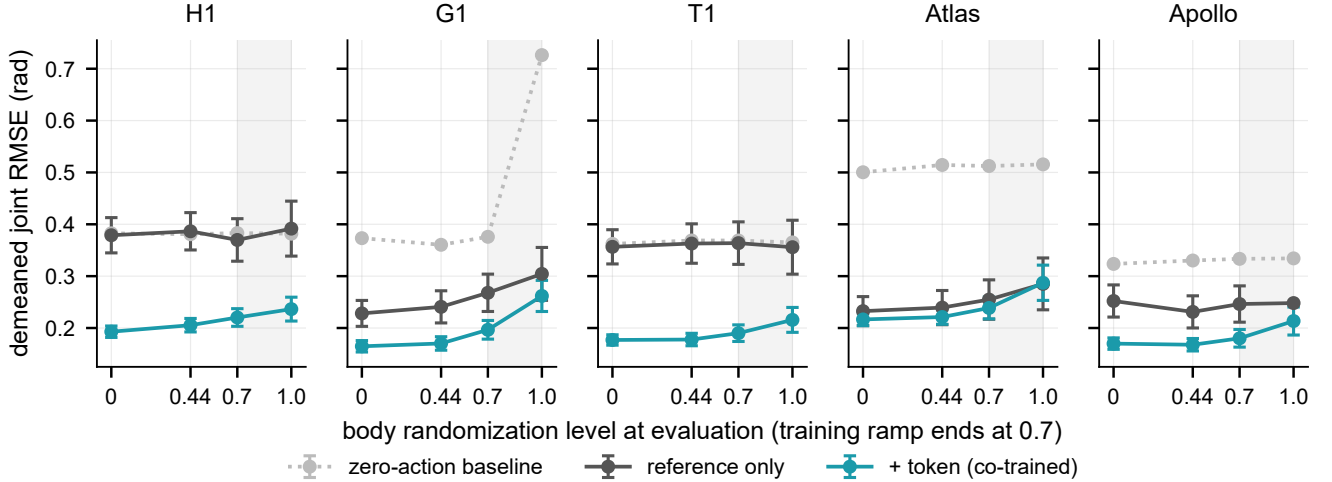


Fig. 11. Five-robot policies evaluated on randomized bodies. Demeaned joint RMSE on the 20 training motions when every evaluation episode samples a body variant at randomization level 0 (nominal), 0.44, 0.7 (the end of the training ramp) or 1.0 (beyond the training range, shaded), for the zero-action baseline, the reference-only policy and the token policy on each robot (mean $\pm$ std over training seeds). The token-policy RMSE increases by 6–19% at the end of the training range and remains 36–64% below the baseline at $c = 1.0$; the reference-only policy remains at the baseline on H1 and T1 at every level.

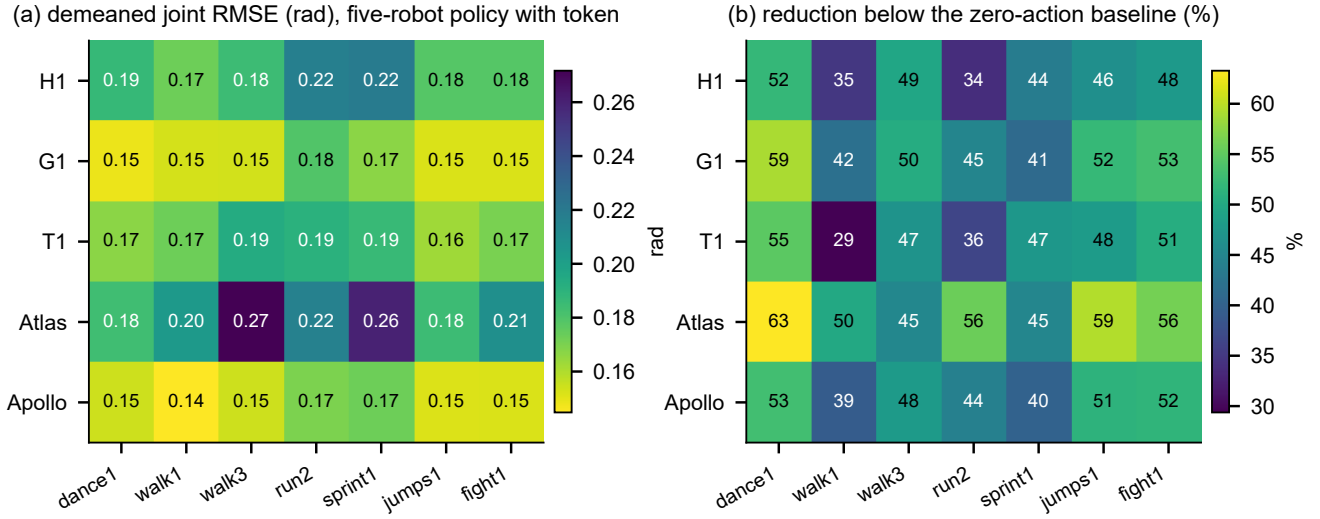


Fig. 12. Every held-out sequence on every robot. (a) Demeaned joint RMSE of the five-robot token policy for each of the 7 held-out LAFAN1 sequences on each robot; (b) the same values as the reduction relative to the zero-action baseline evaluated on the same robot and sequence. The policy stays 29–63 % below the baseline in every combination. Relative to the baseline the reduction is largest on the dance sequence on every robot (52–63 %) and smallest on the walking, running and sprinting sequences (29–56 %).

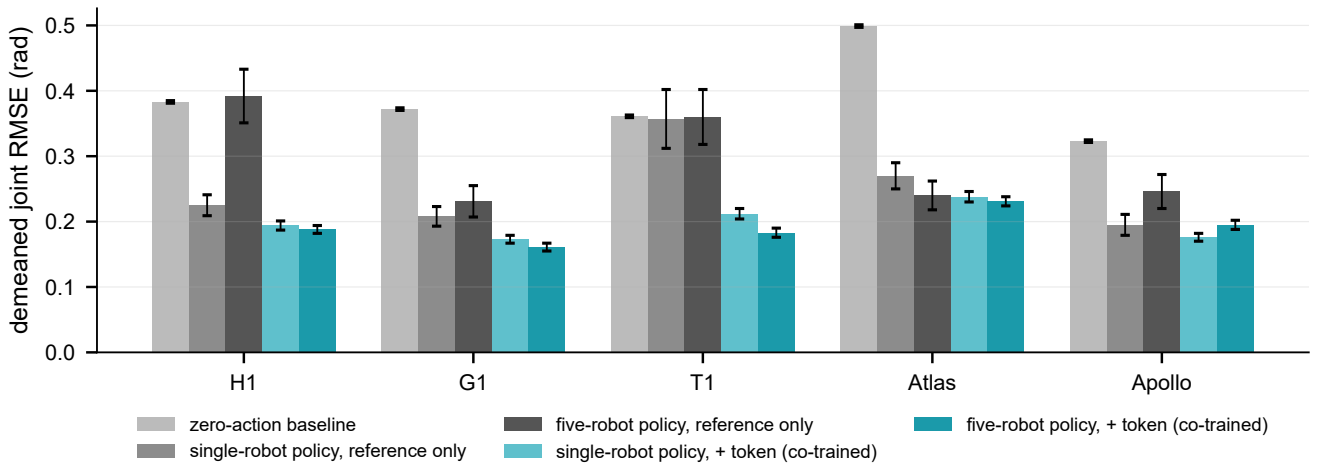


Fig. 13. One shared policy against one policy per robot. Demeaned joint RMSE on the 20 training motions of a single-robot policy trained for 19.7 M steps on that robot (lighter bars) next to the five-robot policy trained for 59 M steps in total (11.8 M per robot); mean $\pm$ std over training seeds, for the reference-only and the token input. With the token input the shared policy has similar or lower mean error on H1, G1, T1 and Atlas, and higher mean error on Apollo. Total training budgets differ, so this is not an equal-compute comparison. Single-robot reference-only policies are themselves unstable under body randomization: on T1 the single-robot policy ends at the baseline.

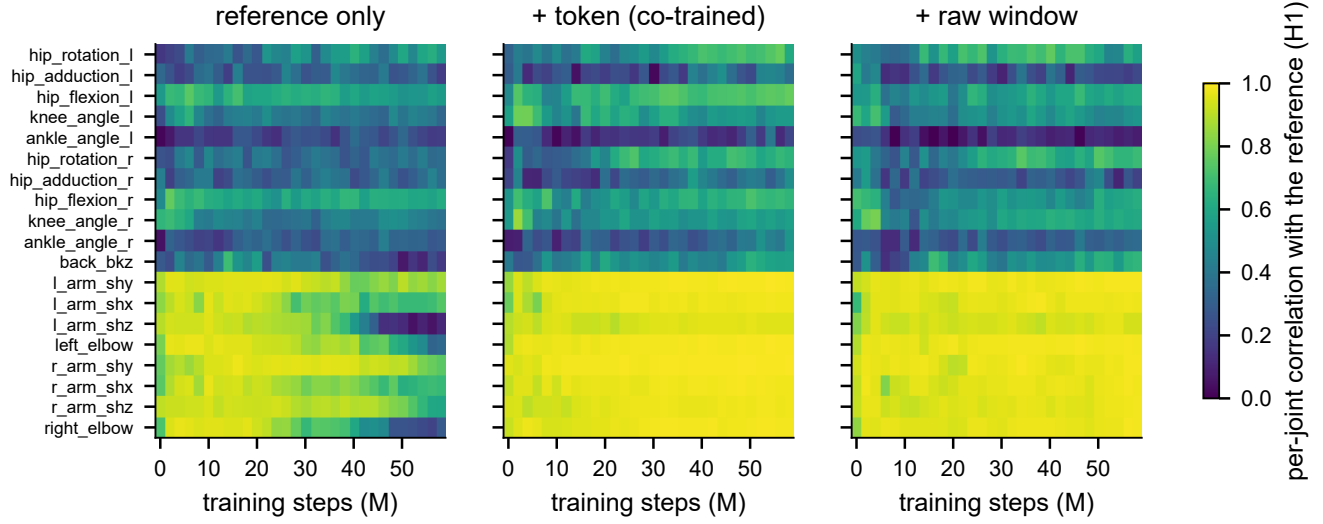


Fig. 14. Per-joint tracking during training: correlation between the executed and the reference trajectory of every H1 joint over training, for the three four-robot policies of Fig. 5. Leg joints correlate weakly with the reference from the start for every policy. Under the reference-only policy, correlation decreases for several arm joints, particularly the elbows and shoulder rotation, while raw and quantized windows maintain high arm-joint correlations.

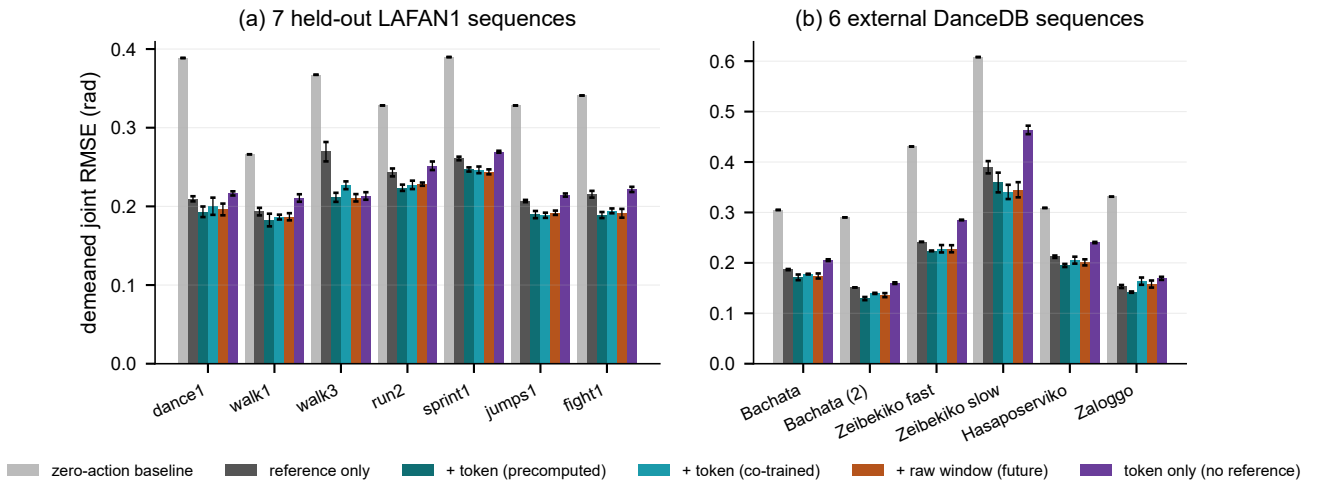


Fig. 15. Cross-motion generalization of the two-robot policies (H1, $1\times$ budget, mean $\pm$ std over training seeds). (a) The seven held-out LAFAN1 sequences, none of which was used for policy training or tokenizer fitting. (b) Six external DanceDB motion-capture sequences. Every policy tracks every held-out sequence below the zero-action baseline. On held-out LAFAN1, window-conditioned policies reduce aggregate error by 9–11 % relative to reference-only control; the token-only policy, which has no direct joint-reference input, matches reference-only control on these sequences.

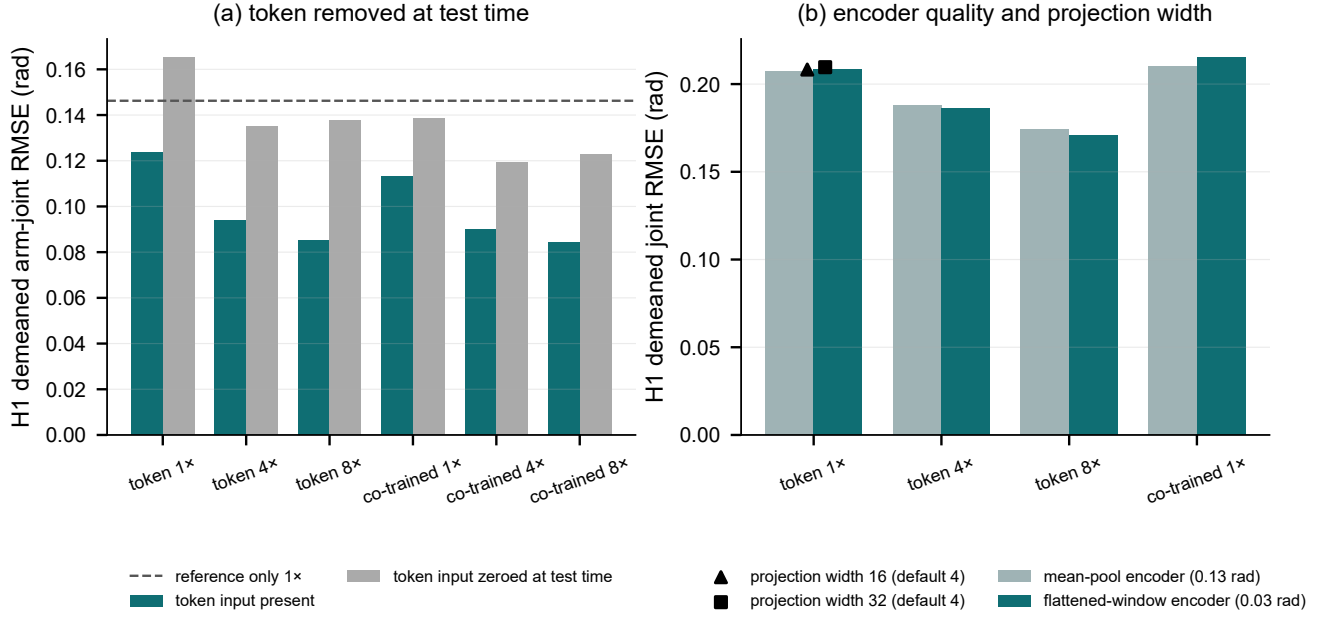


Fig. 16. Encoder and code ablations on H1. (a) Zeroing the token at evaluation increases arm-joint RMSE, showing that the policy uses the token at run time, but does not reproduce the severe reference-only degradation observed at intermediate training budgets. (b) With the direct reference present, flattened-window and mean-pool encoders achieve similar tracking error. At 1x, increasing the token-projection width from 4 to 16 or 32 provides no clear improvement.

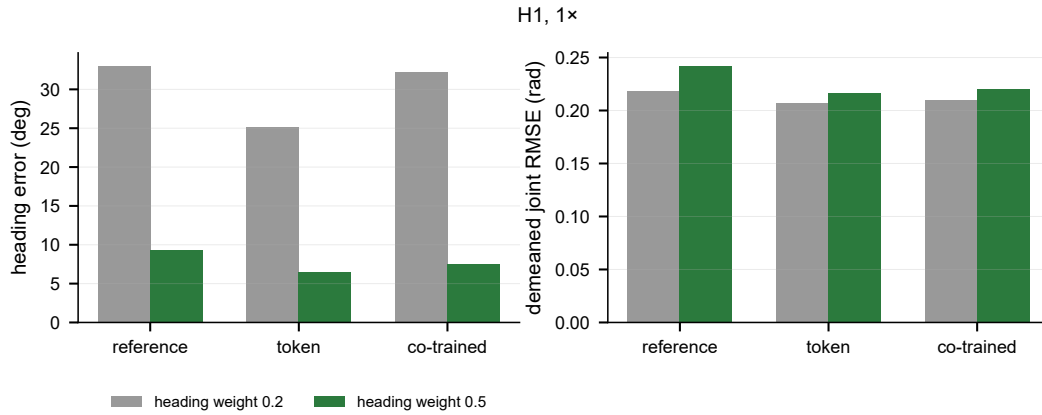


Fig. 17. Increasing the heading-reward weight from 0.2 to 0.5 reduces H1 root-heading error to below 10° for the two-robot policies, with increased joint-tracking error; on the four-robot policy it reduces the error from 45–60° to about 15°.

TABLE VIII

H1 DEMEANED JOINT RMSE IN RADIANS ON THE 20 TRAINING MOTIONS AT THE FULL COMMAND RATE, MEAN $\pm$ STD OVER TRAINING SEEDS (n), WITH ARM-JOINT AND LEG-JOINT VALUES.

Condition	Budget	All	n	Arms	Legs
reference only	1 $\times$	0.219 ± 0.003	4	0.146	0.269
	2 $\times$	0.241 ± 0.014	5	0.214	0.269
	4 $\times$	0.268 ± 0.028	4	0.293	0.255
	8 $\times$	0.201 ± 0.003	5	0.164	0.233
	16 $\times$	0.169 ± 0.000	3	0.113	0.208
+ token (precomp.)	1 $\times$	0.207 ± 0.004	4	0.124	0.261
	2 $\times$	0.199 ± 0.006	3	0.104	0.257
	4 $\times$	0.188 ± 0.002	3	0.094	0.244
	8 $\times$	0.174 ± 0.004	3	0.085	0.226
	16 $\times$	0.152 ± 0.002	3	0.077	0.196
+ token (co-trained)	1 $\times$	0.210 ± 0.005	4	0.113	0.269
	2 $\times$	0.200 ± 0.008	3	0.107	0.257
	4 $\times$	0.182 ± 0.005	3	0.090	0.236
	8 $\times$	0.169 ± 0.002	3	0.084	0.218
	16 $\times$	0.155 ± 0.001	3	0.078	0.200
+ raw window (future)	1 $\times$	0.209 ± 0.002	3	0.120	0.265
	2 $\times$	0.200 ± 0.002	3	0.103	0.259
	4 $\times$	0.187 ± 0.002	3	0.089	0.242
	8 $\times$	0.174 ± 0.002	3	0.083	0.225
	16 $\times$	0.154 ± 0.002	3	0.077	0.201
+ raw window (past)	1 $\times$	0.209 ± 0.003	3	0.110	0.269
	2 $\times$	0.198 ± 0.002	3	0.089	0.258
	4 $\times$	0.187 ± 0.003	3	0.086	0.244
	8 $\times$	0.174 ± 0.002	3	0.084	0.226
	16 $\times$	0.156 ± 0.002	3	0.080	0.203
token only	1 $\times$	0.244 ± 0.001	3	0.172	0.297
	2 $\times$	0.228 ± 0.003	3	0.145	0.285
zero-action baseline	—	0.383	—	—	—

TABLE IX

ROOT-RELATIVE MEAN PER-LINK POSITION ERROR (MPJPE) FOR H1–G1 POLICIES, USING 8 PARALLEL ENVIRONMENTS OVER 1000 CONTROL STEPS. ERRORS ARE COMPUTED AFTER FORWARD KINEMATICS AND AVERAGED OVER LINKS AND FRAMES. REPORTED SUCCESS REQUIRES NO TERMINATION AND MEAN LINK ERROR BELOW 0.5 M AT EVERY EVALUATED FRAME. THIS METRIC EXCLUDES GLOBAL TRANSLATION AND NEED NOT RANK POLICIES IDENTICALLY TO DEMEANED JOINT RMSE. GLOBALLY ALIGNED MPJPE WAS 2–8 M OVER THE 20 S ROLLOUT, DOMINATED BY ROOT DRIFT; PER-POLICY GLOBAL VALUES ARE NOT TABULATED.

Budget	Motion input	MPJPE (mm)		success (%)	
		H1	G1	H1	G1
1 $\times$	reference only	56	58	75	75
1 $\times$	+ token (precomputed)	45	50	100	88
1 $\times$	+ token (co-trained)	53	53	88	75
1 $\times$	+ raw window (future)	45	53	100	62
1 $\times$	+ raw window (past)	48	56	75	88
1 $\times$	token only	53	55	100	100
4 $\times$	reference only	48	59	100	100
4 $\times$	+ token (co-trained)	42	47	100	100
4 $\times$	+ raw window (future)	42	44	100	88

TABLE X

ROOT-RELATIVE MPJPE (MM) OF THE MULTI-ROBOT POLICIES ON THE NOMINAL BODIES (20 TRAINING MOTIONS, ONE TRAINING SEED, 8 ENVIRONMENTS $\times$ 1000 STEPS). † APOLLO IS A HELD-OUT ROBOT MODEL FOR THE FOUR-ROBOT POLICIES; ON IT EVERY POLICY TERMINATES WITHIN THE ROLLOUT, SO ITS NUMBER COVERS THE STEPS BEFORE THE FALL.

Policy	H1	G1	T1	Atlas	Apollo
five-robot, reference only	73	73	63	103	112
five-robot, + token (co-trained)	45	60	42	95	63
four-robot, + token (co-trained) †	46	49	36	83	79
four-robot, + raw window †	45	48	37	88	101
four-robot, token only †	58	62	49	107	136

TABLE XI

TOKENIZER RECONSTRUCTION ERROR ON HELD-OUT MOTIONS FOR DIFFERENT CODE SIZES AND ENCODER ARCHITECTURES; ONE TOKENIZER FIT PER ROW. BIT COUNTS ASSUME BIT-PACKED CODES, WHILE THE RAW 11-FRAME WINDOW USES 1408 BITS PER JOINT WITH 32-BIT VALUES. RECONSTRUCTION ERROR INCREASES AS THE FLATTENED-WINDOW CODE IS REDUCED. THE MEAN-POOL ENCODER DID NOT SUPPORT SUCCESSFUL TOKEN-ONLY TRAINING IN THE TESTED CONFIGURATION.

Encoder	dims $\times$ levels	bits / joint / update	held-out RMSE (rad)
flattened window	32×8	96	0.035
flattened window	16×4	32	0.072
flattened window	8×4	16	0.138
mean-pool	32×8	96	0.148

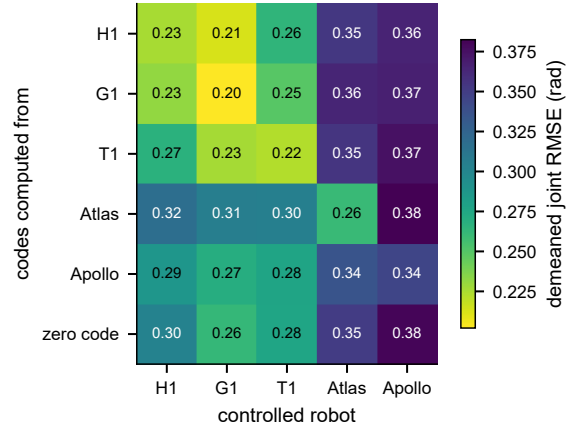


Fig. 18. Preliminary cross-body use of the code: the token-only four-robot policy controls the robot in each column while receiving codes computed from another robot’s retarget of the same walk (rows). Across robots, source codes are assigned to target joints by matching body side and joint function (e.g., hip yaw, knee, shoulder pitch, or elbow); target joints without a corresponding source joint receive a zero code. The most favourable transfers occur among H1, G1 and T1; transfers to Atlas and Apollo are less favourable. This single-motion test supports limited code reuse, not an embodiment-invariant representation.